```
CREATE SCHEMA temp;
```

Within the recreated temp schema, create a slimmed down users table with an id and name column. This is a simplified form of a users table.

Next, populate the table with 10 million rows. Use CREATE TABLE AS to create the table and populate it in one statement.

Use GENERATE_SERIES() to get Integer values from 1 to 10 million for the id and to create a unique name.

Toggle timing to on with \timing to see how long the operation takes.

Run the following statements in psql. This will take a bit of time to populate.

sql/migration_dangerous_defaults_setup.sql

```
-- Enable timing
\timing

-- create users table (id, name)
-- populate it with 10,000,000 rows
CREATE TABLE temp.users AS
SELECT
  seq AS id,
  'Name-' || seq::TEXT AS name
FROM generate_series(1, 10000000) AS t(seq);
```

Verify the users table structure with \d temp.users and confirm it has 10 million rows.

Imagine you've made an application change and want to associate Users to Cities using a city_id column. You want all new and existing Users to have a City assigned by default.

In the application you might prompt users to change their default Cities, but you'd like to start with a default value.

To accomplish this, you'll add a new city_id column and give it a default value. This value will then be used for all new rows being inserted, but will also be applied retroactively to all 10 million existing rows.

For long time PostgreSQL users, adding a Default column value carries some baggage. Column defaults have been made mostly safe to perform on newer versions of PostgreSQL but there are still some gotchas.

Open up a second psql session connected to the Rideshare database. The example below uses "psql1" and "psql2" to refer to the first and second sessions.